



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Sviluppo di un Middleware basato su Spring per l'Arricchimento di Dati

Leonardo Baldo

25 Settembre 2026

2026-09-10

Middleware Spring per Enrichment Dati



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Sviluppo di un Middleware basato su
Spring per l'Arricchimento di Dati

Leonardo Baldo
25 Settembre 2026

1. Buongiorno a tutti.
2. Il lavoro che presento riguarda lo sviluppo di un Middleware basato su Spring per l'arricchimento di dati, realizzato durante il tirocinio presso SyncLab.
3. Vi spiegherò il contesto del progetto, le scelte fatte, e i risultati ottenuti.

- Stage presso **SyncLab S.r.l.**
- Soluzioni informatiche, ricerca e sviluppo, *system integrator*
- Progetto: studio **Middleware** per l'arricchimento di dati



L'azienda

1. Il tirocinio si è svolto presso SyncLab, in modalità prevalentemente remota con presenza settimanale in sede.
2. SyncLab si occupa principalmente di soluzioni informatiche, ricerca e sviluppo e integrazioni di sistemi.
3. Per l'appunto il lavoro consisteva nello sviluppare un Middleware per l'arricchimento di dati, ovvero un software che si colloca tra due o più sistemi, occupandosi di farli comunicare senza che questi debbano conoscersi direttamente.

- Stage presso **SyncLab S.r.l.**
- Soluzioni informatiche, ricerca e sviluppo, *system integrator*
- Progetto: studio **Middleware** per l'arricchimento di dati



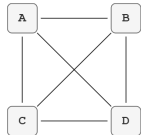
Perché serve un Middleware?



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Problema

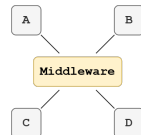
- **Connessioni** dirette con ogni sistema
- Ognuno deve **conoscere** il formato dati dell'altro



Senza Middleware

Soluzione

- Sistemi **eterogenei** che non comunicano direttamente
- **Disaccoppiamento** tra produttori e consumatori



Con Middleware

2026-09-10

Middleware Spring per Enrichment Dati

└ Perché serve un Middleware?

Perché serve un Middleware?

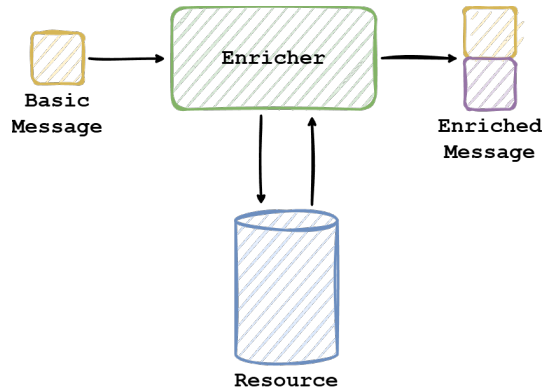
Problema	Soluzione
• Connessioni dirette con ogni sistema • Ognuno deve conoscere il formato dati dell'altro	• Sistemi eterogenei che non comunicano direttamente • Disaccoppiamento tra produttori e consumatori

Senza Middleware

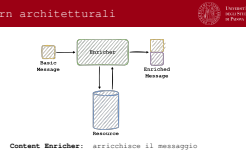
Con Middleware

1. Perché serve un Middleware? Partiamo dal problema. In un sistema con più applicazioni che devono scambiarsi dati, se ogni sistema comunica direttamente con tutti gli altri, il numero di connessioni cresce in modo quadratico, come si vede nello schema a sx, con quattro sistemi servono già sei collegamenti diretti, e ognuno deve conoscere il formato dati dell'altro.
2. Introducendo un Middleware come intermediario, come si vede nello schema a dx, ogni sistema comunica solo con esso: le connessioni diventano lineari, non più dirette, e i sistemi restano disaccoppiati, cioè indipendenti l'uno dall'altro.
3. È esattamente questo il problema che il Middleware sviluppato risolve, occupandosi anche di dati incompleti che devono essere arricchiti con informazioni aggiuntive prima di essere ridistribuiti.

Pattern architetturali

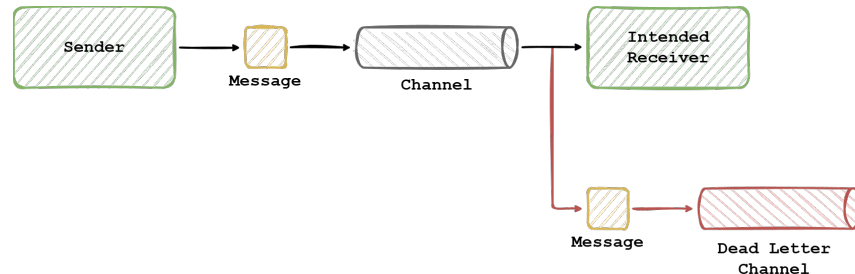
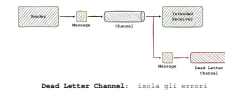


Content Enricher: arricchisce il messaggio



1. Dal punto di vista architetturale, il Middleware implementa un pattern noto come Content Enricher, uno degli Enterprise Integration Patterns: un catalogo di soluzioni standard per problemi ricorrenti nell'integrazione tra sistemi.
2. Il Content Enricher riceve in ingresso un messaggio con informazioni parziali, lo arricchisce interrogando una fonte dati esterna (come per esempio un database), e produce in uscita un messaggio completo.

Pattern architetturali



Dead Letter Channel: isola gli errori

1. Il secondo pattern adottato è il Dead Letter Channel, anch'esso parte degli Enterprise Integration Patterns.
2. Quando un messaggio non può essere arricchito correttamente, ad esempio perché il dato collegato non esiste, questo viene instradato su un canale separato, il Dead Letter Channel, dedicato esclusivamente alla gestione degli errori.
3. Questo consente al flusso principale di proseguire senza interruzioni, mantenendo al contempo traccia dei messaggi problematici per un'analisi o un riprocessamento successivo.

Stack tecnologico



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Linguaggio e piattaforma



Framework applicativo



Infrastruttura di supporto



2026-09-10

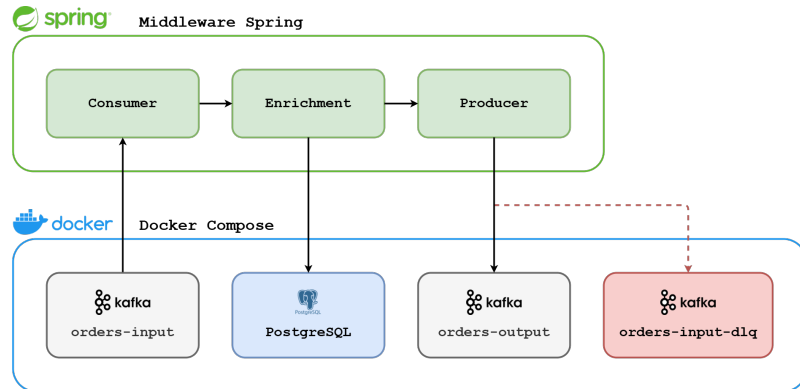
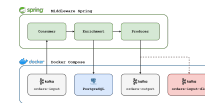
Middleware Spring per Enrichment Dati

Stack tecnologico

1. Concretamente, questi pattern sono stati implementati con Java come linguaggio e Maven per la gestione del progetto e delle sue dipendenze.
2. Come framework applicativo ho usato Spring (basato appunto su Java), in particolare tre suoi moduli: Spring Boot (che semplifica la configurazione e l'avvio dell'applicazione), Spring Data JPA (per l'accesso al database e la gestione dei dati) e Spring Kafka (per la lettura e scrittura dei messaggi).
3. Infine, come infrastrutture di supporto, Docker per containerizzare (cioè eseguire in ambienti isolati e riproducibili), PostgreSQL (come database per contenere i dati per arricchire) e Apache Kafka (per la gestione dei topic).



Il progetto



1. Questo schema riassume l'intera architettura:
 - l'applicazione Spring, in alto, legge, tramite il Consumer, dal topic Kafka orders-input (un topic è semplicemente un canale su cui vengono pubblicati e letti i messaggi);
 - poi tramite l'Enrichment, appunto arricchisce il dato interrogando PostgreSQL per ottenere i dati aggiuntivi;
 - infine, tramite il Producer pubblica il risultato su un altro topic Kafka orders-output, oppure, in caso di errore come visto dal pattern Dead Letter Channel, sul topic orders-input-dlq.
2. Tutta l'infrastruttura in basso, è containerizzata tramite Docker Compose.

Jakarta Persistence:

- **@Entity**
- **@Table, @Id, @Column**

Lombok:

- **@Getter, @Setter**
- **@NoArgsConstructor**
- **@AllArgsConstructor**

```
@Entity
@Table(name = "customers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    @Id
    private Long id;
    @Column(name = "full_name",
            nullable = false)
    private String fullName;
    @Column(name = "email")
    private String email;
    @Column(name = "city")
    private String city;
}
```

Implementazione - un esempio

1. Per darvi un'idea concreta del codice, questa è l'entità che rappresenta la classe cliente.
2. Si può notare l'uso delle annotazioni (un modo, tipico di Java, di aggiungere informazioni al codice senza scriverne di aggiuntivo):
 - quelle di Jakarta Persistence (Entity, Table, Id, Column), sono uno standard Java per definire come una classe si collega a una tabella del database;
 - quelle di Lombok (i metodi Getter, Setter e i costruttori), generano automaticamente in fase di compilazione codice ripetitivo che altrimenti andrebbe scritto a mano, mantenendo la classe concisa.

Implementazione - un esempio

Jakarta Persistence:

- **@Entity**
- **@Table, @Id, @Column**

Lombok:

- **@Getter, @Setter**
- **@NoArgsConstructor**
- **@AllArgsConstructor**

```
@Entity
@Table(name = "customers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    @Id
    private Long id;
    @Column(name = "full_name",
            nullable = false)
    private String fullName;
    @Column(name = "email")
    private String email;
    @Column(name = "city")
    private String city;
}
```

Docker: Topic Kafka, PostgreSQL, Kafka UI

Le **partizioni** permettono letture e scritture in parallelo sullo stesso topic

Topic	Partizioni	Ruolo
orders-input	3	ingresso
orders-output	3	uscita
orders-input-dlq	1	errori



topic

Docker: Topic Kafka, PostgreSQL, Kafka UI

Le **partizioni** permettono letture e scritture in parallelo sullo stesso topic

Topic	Partizioni	Ruolo
orders-input	3	ingresso
orders-output	3	uscita
orders-input-dlq	1	errori



topic

1. L'ambiente di sviluppo è organizzato attorno a tre topic Kafka.
2. Ogni topic ha un numero di partizioni diverso a seconda del ruolo, in particolare i topic del flusso principale ne hanno di più per consentire il parallelismo in lettura/scrittura, la singola partizione per DLQ indica un volume atteso più basso.
3. Il tutto è orchestrato integralmente con Docker, insieme a PostgreSQL e a un'interfaccia web (grazie alla dipendenza Kafka UI) per ispezionare i messaggi, altrimenti visibili tramite riga di comando.



Disaccoppiamento

Sistemi indipendenti,
facilmente modificabili



Errori isolati

Gestiti tramite
Dead Letter Channel



Nessuna perdita dati

Persistenza garantita da Kafka



Ambiente riproducibile

Docker Compose,
un solo comando

Vantaggi ottenuti

1. I vantaggi principali ottenuti sono:
 - Disaccoppiamento: ovvero i sistemi coinvolti restano disaccoppiati
 - Nessuna perdita dati: nessun dato viene perso grazie alla persistenza di Kafka
 - Errori isolati: gli errori vengono isolati automaticamente tramite il DLQ
 - Ambiente riproducibile: ambiente è riproducibile su qualsiasi macchina grazie all'uso di Docker.



- **Apache Flink:** confronto già studiato a livello teorico
- **Test di performance:** metodologia di valutazione già definita, ma non eseguita



Apache Flink



└Sviluppi futuri

1. Gli sviluppi futuri principali sono due:
 - l'implementazione dello stesso progetto tramite Apache Flink, un motore di stream processing che, a differenza di Spring (che elabora un messaggio alla volta non appena arriva) è pensato per elaborare flussi continui di dati in modo nativo. Per limiti di tempo, questo confronto è stato condotto solo a livello teorico e progettuale, senza un'implementazione concreta.
 - l'esecuzione dei test di performance attraverso la metodologia definita nella prossima slide

- **Apache Flink:** confronto già studiato a livello teorico
- **Test di performance:** metodologia di valutazione già definita, ma non eseguita



Sviluppi futuri



Throughput

Messaggi arricchiti
per secondo



Consumer lag

Quanto il sistema
"sta al passo"



Latenza end-to-end

Tempo tra evento grezzo e
arricchito



Costo interrogazione

Tempo di risposta
della fonte dati


Throughput
Messaggi arricchiti
per secondo


Consumer lag
Quanto il sistema
"sta al passo"


Latenza end-to-end
Tempo tra evento grezzo e
arricchito


Costo interrogazione
Tempo di risposta
della fonte dati

1. Ho definito una metodologia per valutare le prestazioni del sistema, seguendo le seguenti metriche:
 - throughput, quanti messaggi vengono elaborati al secondo
 - latenza end-to-end, ovvero il tempo tra la pubblicazione di un evento grezzo sul topic sorgente (orders-input) e la comparsa del corrispondente evento arricchito sul topic di destinazione (orders-output)
 - consumer lag, che indica quanto il sistema è indietro rispetto ai messaggi in arrivo
 - costo interrogazione, quindi il tempo di risposta della fonte dati esterna
2. Anche in questo caso l'esecuzione concreta dei test non è stata eseguita, quindi l'implementazione di Apache Flink e l'esecuzione dei test di performance restano il prossimo passo.

Grazie per l'attenzione

Domande?

1. Vi ringrazio per l'attenzione, la mia discussione finisce qui.